

Software Performance and Scaling

From Profiling and Caching to Algorithmic Complexity

Martin Benning

University College London

ENGF0034 – Design and Professional Skills

Lecture Overview

Objectives

Today we tackle the "difficult question" of software performance. We will explore diagnosis techniques, the impact of language choice, the realities of hardware memory hierarchies, and how to scale software when data volume explodes.

- 1 The Performance Problem
- 2 Step 1: Diagnosis and Profiling
- 3 Step 2: The Language Factor
- 4 Step 3: The Hardware Reality
- 5 Step 4: Algorithmic Complexity at Scale
- 6 Step 5: Modern Scaling and Hardware Trends
- 7 Summary

The Performance Problem

The Performance Problem

Why is my Code Slow?

Performance issues are often subtle and difficult to diagnose. Common causes include:

- **Inefficient Implementation:** The logic is simply written poorly.
- **Inefficient Language:** Using an interpreted language (like Python) for compute-heavy tasks.
- **Poor Algorithmic Complexity:** Using $O(N^2)$ when $O(N \log N)$ is required.
- **I/O Latency:** Spending too much time waiting for disk or network data.
- **Concurrency Issues:** Waiting on thread locks.
- **Data Volume:** Simply having too much data for a single machine.

The Golden Rule of Optimisation

The worst thing you can do is jump in and try to fix things without evidence.

Donald Knuth's Wisdom

"Premature optimisation is the root of all evil (or at least most of it) in programming."

Nuance

Knuth did not mean "never optimise." He meant that programmers spend too much time worrying about efficiency in the *wrong places* and at the *wrong times*.

Step 1: Diagnosis and Profiling

Step 1: Diagnosis and Profiling

Don't Guess, Measure

To fix performance, we must identify exactly where the code spends its time.

- In previous lectures, we saw that popping from the front of a list is $O(N)$, while popping from the back is $O(1)$.
- We verified this using **profiling**.

The Tool: cProfile

Python comes with a built-in profiler called `cProfile`. It measures execution time per function call.

Using cProfile I

To profile a Python script, we can instantiate a profiler around the critical code section.

Listing 1: Profiling a Tetris Game

```
1 import cProfile
2 import pstats
3
4 # Instantiate the profiler
5 profiler = cProfile.Profile()
6 profiler.enable()
7
8 # --- Run the code you want to measure ---
9 game.run()
10 # -----
```


Using cProfile II

```
11 profiler.disable()
12
13
14 # Process and print stats
15 stats = pstats.Stats(profiler).strip_dirs().sort_stats('cumulative'
16               )
17 stats.print_stats()
```

Visualizing Profiles

Raw text output from profilers can be difficult to interpret. Tools like **SnakeViz** turn profiling data into "Flame Graphs".

Flame Graph Analysis

- **Width:** Represents the proportion of total time spent in a function.
- **Depth:** Represents the call stack (who called whom).
- **Goal:** Look for wide bars (bottlenecks). In this Tetris agent example, the function `score_position` was consuming excessive time re-initializing data structures.

The Result

By moving initialisation out of the loop, performance improved by 25%. Further algorithmic pruning doubled the speed again.

Modern Context: Next-Gen Profiling

While 'cProfile' is standard, modern tools provide deeper insights.

Scalene (Modern Tool)

cProfile only measures time. Modern tools like **Scalene** distinguish between:

- **Python Time:** Time spent in the interpreter (slow).
- **Native Time:** Time spent in C extensions (NumPy, PyTorch) (fast).
- **System Time:** Waiting for I/O.

Why this matters

If your code spends 90% of its time in "Native", rewriting it in C won't help—it's already running in C!

Step 2: The Language Factor

Step 2: The Language Factor

Why is Python Slow?

If profiling shows the bottleneck is in Python code, we must understand why.

1. Interpreted Nature

Python reads and interprets code line-by-line rather than compiling it directly to machine instructions beforehand.

2. Safety and Dynamic Typing

- **Integer Overflow:** Python automatically handles arbitrarily large integers, whereas C requires manual management.
- **Bounds Checking:** Python checks list indices; C might just segfault or return garbage.

Solution A: JIT Compilation (PyPy)

We don't always need to rewrite code. We can use a Just-In-Time (JIT) compiler like **PyPy**.

- PyPy replaces the standard Python interpreter.
- It compiles frequently used code paths into machine code at runtime.

Benchmark Results

- Standard Python (Interpreted): 13 seconds.
- PyPy (Compiled): 4.4 seconds.
- **Result:** $> 2\times$ speedup with zero code changes.

Modern Context: Python 3.11+ and 3.13

The "Python is slow" narrative is changing rapidly.

Python 3.11+

Recent versions of standard Python have integrated adaptive specialisation, resulting in up to 60% speedups over older versions without external tools.

Python 3.13 (Experimental)

- **JIT:** Now includes an experimental JIT compiler in the standard build.
- **No-GIL:** An experimental build removes the Global Interpreter Lock, allowing true multi-core parallelism in pure Python for the first time.

Solution B: Native Extensions

For maximum performance, we can rewrite critical sections in a lower-level language.

- **C/C++:** The traditional approach. Libraries like `pygame` use C for graphics rendering while leaving game logic in Python.
- **Rust:** Modern, memory-safe alternative to C.
- **Mojo:** A modern superset of Python designed for AI hardware, bridging the gap between Python ease-of-use and C performance.

Step 3: The Hardware Reality

Step 3: The Hardware Reality

The Memory Wall

If code is compiled and optimised but still slow, the bottleneck is often **Memory Latency**.

The CPU-Memory Gap

CPUs have become exponentially faster, but memory (RAM) access speeds have not kept pace.

- **CPU Clock:** ≈ 0.3 nanoseconds per cycle (3 GHz).
- **RAM Access:** ≈ 100 nanoseconds.

Fetching data from RAM effectively halts the CPU for hundreds of cycles.

The Cache Hierarchy

To hide this latency, hardware architects use layers of Cache.

- **L1 Cache:** Tiny (32KB), Instant access (≈ 4 cycles).
- **L2 Cache:** Small (1MB), Fast (≈ 10 cycles).
- **L3 Cache:** Shared (16MB), Moderate (≈ 40 cycles).
- **Main Memory (DRAM):** Huge (GBs), Slow (200 — 300 cycles).

Locality of Reference

Performance depends heavily on whether your data is in the cache when the CPU needs it.

Spatial Locality

When the CPU loads a byte, it loads a whole "cache line" (typically 64 bytes).

- If you access data sequentially (e.g., an array '[A, B, C, D]'), loading 'A' also pulls 'B, C, D' into the fast cache.
- If you access scattered data (e.g., a linked list with nodes scattered in RAM), every access is a cache miss (slow).

Design Implication

Data structure layout matters. Arrays often outperform Linked Lists simply due to cache locality, even if the theoretical Big-O complexity is the same.

Real World Example: Netflix

Optimizing Video Serving

A research team at Cambridge designed a video server structure (similar to Netflix) focusing on cache locality.

- They ensured the data path never touched main DRAM, staying within the L3 Cache.
- **Result:** A 4-core server outperformed a 12-core server.

Lesson

Hardware-aware programming (Cache Locality) can yield order-of-magnitude improvements.

Step 4: Algorithmic Complexity at Scale

Step 4: Algorithmic Complexity at Scale

When Optimisation Isn't Enough

Micro-optimisations (caches, compilers) cannot fix a fundamentally bad algorithm.

The Scale Problem

If your algorithm is $O(N^2)$ and you increase data by $1000\times$:

- Runtime increases by 1,000,000.
- No amount of C++ or Cache Locality will save you.

Example: Network Simulation

Consider a simulation requiring the processing of hundreds of thousands of events.

- **Problem:** Storing events in a single binary heap ($O(\log N)$) is too slow due to the size of N .
- **Solution:** Split the data structure. Only store "head-of-line" events in the main heap.
- **Result:** N reduced from 100k to 10k. Combined with better cache usage, performance

When Optimisation Isn't Enough

The Scale Problem

If your algorithm is $O(N^2)$ and you increase data by $1000\times$:

- Runtime increases by 1,000,000.
- No amount of C++ or Cache Locality will save you.

Example: Network Simulation

Consider a simulation requiring the processing of hundreds of thousands of events.

- **Problem:** Storing events in a single binary heap ($O(\log N)$) is too slow due to the size of N .
- **Solution:** Split the data structure. Only store "head-of-line" events in the main heap.
- **Result:** N reduced from 100k to 10k. Combined with better cache usage, performance doubled.

Case Study: Hacker News

How do you serve millions of requests on modest hardware?

Architecture

- **Traffic:** 300k unique visitors/day, 3M page views.
- **Hardware:** Single server, single core execution.
- **Language:** Arc (a dialect of Lisp).

Takeaway

They didn't need a massive distributed cluster. They needed the *correct* algorithmic complexity for lookups and data generation.

Step 5: Modern Scaling and Hardware Trends

Step 5: Modern Scaling and Hardware Trends

The End of Frequency Scaling

Moore's Law vs. Reality

Moore's Law (transistor count doubling) continues, but **Dennard Scaling** (frequency increasing) stopped around 2005.

- **1970–2000:** CPUs got faster every year (Clock speed).
- **2000–Present:** Clock speeds capped at $\approx 3 - 5$ GHz due to thermal limits.

So, how do we use the extra transistors?

Where do the Transistors Go?

Since we can't make the clock faster, we add:

- 1 **Larger Caches:** (Diminishing returns).
- 2 **Instruction Level Parallelism (ILP):** Doing more math per cycle (Superscalar execution).
- 3 **More Cores:** Parallel processing (Subject of the next lecture).
- 4 **Specialised Hardware:** (Modern Context).

Modern Context: Specialisation (Accelerators)

In the last decade, performance has shifted from general-purpose CPUs to domain-specific accelerators.

GPUs and TPUs

- Instead of complex caching and prediction (CPU), these chips devote transistors to thousands of simple arithmetic cores.
- Essential for AI, Machine Learning, and scientific simulation.
- **Trade-off:** Data must be moved from CPU memory to GPU memory (the bottleneck).

Unified Memory (e.g., Apple Silicon)

Modern SoCs share memory between CPU and GPU to reduce the cost of this data movement.

Summary

Summary

Summary

Key Takeaways

- **Measure, Don't Guess:** Use profilers (cProfile, Scalene) to find the bottleneck.
- **Language Matters:** Python is safe but slow. Compilers (PyPy), native extensions (C/Rust), or modern Python (3.11+) help.
- **Hardware Reality:** Memory is slow. Cache locality (spatial and temporal) is critical for high performance.
- **Algorithmic Complexity:** $O(\log N)$ beats $O(N)$ regardless of the hardware.
- **The Future:** Performance now comes from parallelism (multi-core) and specialization (GPUs), not just faster clock speeds.